

GUIDE-001：项目目录结构与开发规范

融合实习开发清单 + 总需求 + 技术选型

万能产品小助手 (PM AI)
魏子政审阅

2026-05-06 / 版本 v1.0

文档性质	项目开发规范 (Guide)
来源文档	intern-dev-checklist.tex + DEMAND-000-v0-master.tex + REF-002-project-master-plan.tex
当前阶段	V0 先备完成, V1 目录框架就绪

摘要

本文件由 PM AI 编排，定义项目完整目录结构、前后端分层规范、各目录职责、编码约定。融合了《实习开发清单》中的任务分解、技术选型中的组件决策、以及总需求中的业务模块划分。Cursor Agent 在后续开发中应严格遵循本文件定义的目录结构和命名约定。

目录

1 项目总览	3
1.1 目录分类	3
2 后端目录结构	3
2.1 后端分层架构	5
2.2 后端编码规范	5
3 前端目录结构	5
3.1 前端模块划分	6
3.2 前端编码规范	6
4 业务模块与优先级	7
4.1 P0: 核心功能 (MVP)	7
4.2 P1: 重要功能	7
4.3 P2: 增强功能	7
5 数据库相关	7
6 常用命令速查	7

项目总览

enterprise-app-store/	
backend/	# 后端 (FastAPI + SQLAlchemy + Alembic)
frontend/	# 前端 (Vue 3 + Element Plus + Pinia)
database/	# 数据库设计文档 (ER 图、建表 DDL)
reference/	# 参考文档 (调研、规范、预研)
demands/	# 需求文档 (MASTER + DEMAND-xxx)
skills/	# OpenClaw 智能体 Skill 定义
.cursor/rules/	# Cursor 规则文件 (.mdc)
CLAUDE.md	# Cursor 全局指令
AGENTS.md	# OpenClaw 全局指令
docker-compose.yml	# Docker 编排 (PG + MinIO + OpenSearch + Backend)
.env.example	# 环境变量模板

目录分类

目录	性质与维护者
backend/	代码目录, Cursor 负责, PM 审计
frontend/	代码目录, Cursor 负责, PM 审计
database/	设计文档, PM 编排, Cursor 参考
reference/	项目文档, PM 维护
demands/	需求文档, PM 维护
skills/	智能体定义, PM 维护
.cursor/rules/	Cursor 规则, PM 编排

后端目录结构

backend/	
app/	
__init__.py	# 包初始化
main.py	# FastAPI 入口, CORS、路由挂载、生命周期
config.py	# Pydantic Settings, 读 .env
database.py	# SQLAlchemy async engine + session 工厂
models/	# ORM 模型 (每张表一个文件)
__init__.py	# 统一导出所有模型 (供 Alembic 使用)
user.py	# 用户表
app.py	# 应用表
review.py	# 审批记录表
scan.py	# 扫描结果表 (JSONB)
audit_log.py	# 审计日志表
schemas/	# Pydantic 数据模型 (请求/响应)
__init__.py	

```

    user.py                # UserCreate, UserResponse, Token
    app.py                 # AppCreate, AppUpdate, AppResponse
    review.py              # ReviewCreate, ReviewResponse
    scan.py                # ScanReport
    common.py              # PageParams, PageResponse 等通用模型
routers/                  # API 路由 (按功能分文件)
    __init__.py            # 统一挂载
    auth.py                # POST /api/auth/login, /register, /refresh
    apps.py                # CRUD /api/apps
    reviews.py             # 审批 /api/reviews
    search.py              # 搜索 /api/search
    scans.py               # 扫描 /api/scans
    users.py               # 用户管理 /api/users (管理员)
    audit.py               # 审计日志 /api/audit
services/                 # 业务逻辑层 (核心业务代码)
    __init__.py
    auth_service.py        # 认证逻辑 (JWT 签发/验证、密码哈希)
    app_service.py         # 应用 CRUD 逻辑
    review_service.py      # 审批状态流转
    search_service.py      # OpenSearch 搜索逻辑
    scan_service.py        # 三层扫描调度 (Sengrep + ClamAV + LLM)
    minio_service.py       # 文件上传/下载
    audit_service.py       # 审计日志记录
tasks/                    # asyncio 后台任务
    __init__.py
    scan_task.py           # 扫描任务异步执行
    index_task.py          # OpenSearch 索引同步
utils/                    # 工具函数
    __init__.py
    security.py            # JWT 工具、密码哈希
    pagination.py          # 分页工具
    deps.py                # FastAPI Depends (get\_db, get\_current\_user)
tests/                    # 测试文件 (镜像 app/ 结构)
    __init__.py
    conftest.py            # pytest fixtures (测试数据库、客户端)
    test_auth.py
    test_apps.py
    test_reviews.py
    test_scans.py
alembic/                  # 数据库迁移
    env.py                 # Alembic 配置 (async)
    script.py.mako         # 迁移脚本模板
    versions/              # 迁移文件 (自动生成)
requirements.txt           # Python 依赖
Dockerfile                # 后端容器化
.env.example               # 后端专用环境变量
.gitignore

```

后端分层架构

层	目录	职责
路由层	<code>routers/</code>	接收 HTTP 请求，参数校验，调用 <code>service</code> ，返回响应。不写业务逻辑。
业务层	<code>services/</code>	核心业务逻辑。所有数据库操作、外部服务调用都放这里。
数据层	<code>models/</code> + <code>database.py</code>	ORM 模型定义 + 数据库连接。
模型层	<code>schemas/</code>	Pydantic 模型，定义请求/响应格式。
任务层	<code>tasks/</code>	异步后台任务（扫描、索引同步等耗时操作）。
工具层	<code>utils/</code>	通用工具函数（安全、分页、依赖注入）。

后端编码规范

1. 所有接口用 `async/await`
2. 数据校验用 Pydantic，不手动校验
3. 业务逻辑放 `services/`，Router 里只做参数校验和调用
4. 敏感信息放环境变量（`.env`），不硬编码
5. 每个 Router 加 `tags`，方便 Swagger 文档
6. 每个 Router 加 `prefix="/api/xxx"`，统一前缀
7. ORM 模型用 SQLAlchemy 2.0 声明式风格（`DeclarativeBase`）
8. 数据库操作通过 `AsyncSession`，不用同步模式

前端目录结构

```
frontend/
  src/
    main.ts                # 应用入口（挂载 Vue + Element Plus + Router + Pinia）
    App.vue                # 根组件（Layout + RouterView）
    router/
      index.ts             # 路由配置（admin/ 和 portal/ 两套）
      guards.ts            # 路由守卫（登录检查、角色检查）
    views/
      admin/
        apps/              # 应用管理（列表、详情、编辑）
        reviews/          # 审批管理（待审批列表、审批详情）
        users/             # 用户管理（列表、角色分配）
        dashboard/         # 管理首页（统计概览）
      portal/
        explore/           # 应用市场（搜索、浏览、分类筛选）
        submit/            # 提交应用（上传表单）
```

```

    profile/                # 开发者主页（已发布应用、个人信息）
    my-apps/                # 我的应用（版本管理、状态追踪）
  auth/                    # 认证页面
    Login.vue              # 登录
    Register.vue           # 注册
  components/
    ui/                    # 基础 UI 组件（基于 Element Plus 封装）
    AppCard.vue            # 应用卡片
    SearchBar.vue          # 搜索栏（全文 + 语义切换）
    AppUpload.vue          # 应用上传组件
    ReviewPanel.vue        # 审批操作面板
    StatusBadge.vue        # 状态标签（P0/P1/P2 颜色编码）
    ScanReport.vue         # 扫描报告展示
  stores/
    auth.ts                # 用户认证状态（Token、用户信息）
    apps.ts                # 应用列表状态
    search.ts              # 搜索状态
  api/
    request.ts             # axios 实例（baseUrl、拦截器、Token 刷新）
    auth.ts                # 认证 API
    apps.ts                # 应用 API
    reviews.ts            # 审批 API
    search.ts              # 搜索 API
    users.ts               # 用户 API
  utils/
    format.ts              # 格式化工具（日期、文件大小）
    constants.ts           # 常量定义（状态映射、角色枚举）
  styles/
    variables.scss         # CSS 变量（颜色、间距）
    global.scss            # 全局样式
  public/
    favicon.ico
  index.html
  vite.config.ts           # Vite 配置（API 代理）
  tsconfig.json
  tsconfig.app.json
  tsconfig.node.json
  package.json
  .gitignore

```

前端模块划分

前端编码规范

1. 组件用 TypeScript，文件名 PascalCase（AppCard.vue，不是 app-card.vue）
2. 遵循 `<script setup lang="ts">` 风格
3. UI 组件优先用 Element Plus，不自造轮子
4. API 调用统一放 `src/api/`，组件通过 store 间接调用

模块	说明
views/admin/	管理后台：应用管理、审批管理、用户管理、数据概览。仅管理员/审批人可访问。
views/portal/	开发者门户：应用市场、提交应用、开发者主页、我的应用。所有注册用户可访问。
views/auth/	认证页面：登录、注册。未登录用户重定向到此处。
components/	可复用组件：AppCard、SearchBar、StatusBadge 等。按功能命名，Pascal-Case。
stores/	Pinia 状态管理：按业务模块分 store（auth、apps、search）。
api/	API 调用封装：统一 axios 实例 + 按功能模块分文件。组件不直接调 axios。

5. 状态管理用 Pinia，按功能模块分 store
6. 路由守卫做登录检查和角色权限检查
7. 样式用 SCSS + CSS 变量，支持后续暗色模式扩展

业务模块与优先级

以下优先级划分融合了实习开发清单的任务分解和总需求中的业务模块：

P0：核心功能（MVP）

编号	模块	涉及目录	说明
F-001	用户认证	auth router/service + Login.vue	JWT 登录/注册/Token 刷新
F-002	应用 CRUD	apps router/service + admin/apps/	创建/读取/更新/删除应用
F-003	应用列表页	portal/explore/ + AppCard.vue	卡片式展示，分页、筛选、排序
F-004	应用详情页	portal/explore/	版本历史、安装说明
F-005	文件上传	minio_service + AppUpload.vue	应用包上传至 MinIO
F-006	数据库建表	models/ + alembic/	核心表结构 + 种子数据

P1：重要功能

P2：增强功能

数据库相关

database/ 目录存放数据库设计文档（ER 图、建表 DDL），供 Cursor 参考。

常用命令速查

编号	模块	涉及目录	说明
F-007	审批工作台	reviews router/service + admin/reviews/	提交-待审批-通过/驳回-上架/退回
F-008	搜索功能	search service + portal/explore/	OpenSearch 全文 + 语义搜索
F-009	安全扫描	scan service + tasks/	Semgrep + ClamAV + LLM 三层扫描
F-010	RBAC 权限	utils/deps.py + admin/users/	管理员/审批人/开发者/普通用户
F-011	用户管理页	users router + admin/users/	用户列表、角色分配
F-012	OpenSearch 索引	tasks/index_task.py	应用数据同步到 OpenSearch

编号	模块	涉及目录	说明
F-013	评分与评论	组件 + router	星级评分 + 文字评论
F-014	开发者主页	portal/profile/	开发者信息 + 已发布应用
F-015	暗色模式	styles/variables.scss	亮色/暗色切换
F-016	审计日志	audit router/service	记录所有关键操作
F-017	搜索建议	search service + SearchBar.vue	输入时自动补全
F-018	健康检查增强	main.py	/health 返回各组件状态

```

# === 后端 ===
pip install -r backend/requirements.txt          # 安装依赖
uvicorn app.main:app --reload --app-dir backend  # 启动开发服务器
alembic upgrade head --cwd backend               # 运行数据库迁移
alembic revision --autogenerate -m "xxx" --cwd backend # 生成迁移

# === 前端 ===
cd frontend && npm install                        # 安装依赖
cd frontend && npm run dev                         # 启动开发服务器
cd frontend && npm run build                       # 生产构建

# === Docker ===
docker compose up -d                             # 启动所有服务
docker compose down                              # 停止所有服务
docker compose logs backend -f                   # 查看后端日志

# === 测试 ===
cd backend && pytest                              # 运行后端测试
cd backend && pytest --cov=app                    # 带覆盖率

```


存储	用途
PostgreSQL（关系表）	用户、应用元数据、审批记录、权限配置
PostgreSQL（JSONB）	扫描结果、规则命中详情、引擎原始输出
MinIO（对象存储）	应用包文件、头像、附件
OpenSearch（搜索引擎）	全文检索 + 语义搜索索引

更新记录

版本	日期	变更内容
v1.0	2026-05-06	初始版本：融合 intern-dev-checklist + 总需求 + 技术选型，定义完整目录框架